المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ ⵜⵓⵏⵏⵓⵏⵉⵏ - ⴱⵏⵉ ⵎⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

RAPPORT TP1 Génie Logiciel

Cycle Ingénieur filière IA-Cybersécurité

Nom : **OURO-AGORO Yasmine**

Apogée **: 23013788**

1. Introduction

Ce TP a pour objectif la mise en place d'une application monolithique robuste en utilisant le framework **Spring Boot**. L'application vise à gérer un catalogue de produits via une **API REST CRUD** complète, tout en respectant une architecture en couches (Entity, Repository, Service, Controller).

2. Environnement de Travail et Outils

Pour la réalisation de ce projet, les outils suivants ont été utilisés :

- **Langage :** Java (JDK 17+)
- **Gestionnaire de dépendances :** Maven 3.8+
- **IDE :** IntelliJ IDEA / VS Code
- **Base de données :** PostgreSQL
- **Tests API :** Postman

3. Architecture et Implémentation

3.1. Initialisation et Configuration

Le projet a été généré via *Spring Initializr* avec les dépendances clés : **Spring Web**, **Spring Data JPA**, **Lombok**, et **Validation**. La connexion à la base de données a été configurée dans le fichier application.properties pour pointer vers une base nommée ecommerce.

3.2. Modèle de Données (Entité Product)

L'entité Product a été définie avec des validations strictes pour garantir l'intégrité des données :

- **id :** Clé primaire auto-générée.
- **name :** Obligatoire (NotBlank).
- **price :** Doit être positif et non nul.
- **stock :** Ne peut pas être négatif. L'utilisation de **Lombok** (@Getter, @Setter, @Builder) a permis de réduire considérablement le code "boilerplate".

3.3. Couche Accès aux Données (Repository)

Deux approches ont été explorées :

1. Approche Classique **:** Extension de JpaRepository pour bénéficier des méthodes standards et de requêtes personnalisées comme findByNameContainingIgnoreCase.
2. Approche Automatisée **:** Utilisation de @RepositoryRestResource pour générer automatiquement les points de terminaison REST sans contrôleur additionnel.

3.4. Couche Métier (Service)

La classe ProductService centralise la logique métier. Elle utilise l'annotation @Transactional pour assurer l'atomicité des opérations, notamment pour la création, la mise à jour et la suppression des produits.

3.5. Couche Web (Controller)

Le ProductController expose les ressources via l'URL /api/products. Il gère les requêtes HTTP (GET, POST, PUT, DELETE) et assure la validation des données entrantes via @Valid.

4. Tests et Résultats

Les tests ont été effectués avec **Postman** pour valider le bon fonctionnement des endpoints:

Méthode	URL	Description
GET	/api/products	Liste de tous les produits
POST	/api/products	Création d'un nouveau produit
PUT	/api/products/{id}	Modification d'un produit existant
DELETE	/api/products/{id}	Suppression d'un produit

5. Conclusion

Ce TP a permis de maîtriser les fondamentaux d'une application **Spring Boot** monolithique. L'utilisation de **Spring Data JPA** et des **RestRepositories** démontre la puissance du framework pour accélérer le développement tout en maintenant une structure de code propre et évolutive.